

Customer Registry

A MERN Stack Customer Support & Complaint Management Platform
Project Documentation — Full Stack Development (MERN)

1. Introduction

Project Title

Customer Registry

Team Members

Vijayendra Bylupati – Full Stack Developer (Frontend, Backend, Database)

Introduction

The Customer Registry Management System is a full-stack web application built using the MERN stack (MongoDB, Express.js, React, and Node.js) to streamline customer support and complaint handling. The platform supports three distinct user roles — Customer, Agent, and Admin — each with a dedicated dashboard and permissions. Customers can raise complaints and track their resolution, agents can manage and respond to assigned complaints, and administrators oversee users, categories, and overall system analytics. The application features a modern, responsive interface with dark mode support, real-time-style chat between customers and agents, and an in-app notification system.

2. Project Overview

Purpose

The purpose of this project is to provide organisations with a centralised platform to receive, track, and resolve customer complaints efficiently, replacing informal or manual complaint tracking with a structured, role-based digital workflow.

Objectives

- Enable customers to submit and track complaints with file attachments.
- Allow agents to manage assigned complaints and communicate with customers.
- Provide administrators with tools to manage users, agents, and complaint categories.
- Maintain a complete audit trail of every complaint through a status timeline.
- Deliver a secure, role-based, and responsive user experience.

Main Features

- Role-based authentication and dashboards for Customer, Agent, and Admin.
- Complaint creation with category, priority, and optional file attachment.
- Complaint status tracking (open → in progress → resolved/closed) with timeline history.
- In-app chat between customer and assigned agent (REST-based polling).
- Real-time-style in-app notifications for status changes and assignments.

- Post-resolution customer feedback and rating.
- Admin analytics dashboard with charts (Chart.js) for complaint statistics.
- Dark mode toggle with persistence, search, filtering, and pagination.

3. System Architecture

Frontend

The frontend is built with React (using Vite as the build tool) and styled with Tailwind CSS in a glassmorphism / gradient theme. Key libraries include React Router DOM for client-side routing, Axios for API communication, React Icons for iconography, Framer Motion for animations, react-toastify for notifications, and Chart.js (via react-chartjs-2) for the admin analytics charts. Application state such as authentication, theme, and toast messages is managed through React's Context API (AuthContext, ThemeContext, ToastContext).

Backend

The backend is built with Node.js and Express.js, exposing a RESTful API. It follows an MVC-style structure with separate controllers for authentication, customers, agents, complaints, categories, messages, notifications, and users. Route protection and role-based access control are handled through custom middleware (auth.js), which verifies JWT tokens and restricts access based on user role (customer, agent, admin). File uploads (complaint attachments) are handled using Multer, and centralised error handling is implemented via an errorHandler middleware.

Database

MongoDB is used as the database, accessed through Mongoose ODM. The main collections/models are:

- User — stores name, email, hashed password, role (customer/agent/admin), and profile details.
- Complaint — stores title, description, category, priority, status, attachment, feedback, and a status timeline.
- Category — complaint categories such as Billing, Technical Support, Account, Product Feedback, and Other.
- Agent / Customer — role-specific supporting collections.
- Message — chat messages exchanged between customer and agent on a complaint.
- Notification — in-app notifications generated on status changes and assignments.

4. Setup Instructions

Prerequisites

- Node.js (v18 or above)
- MongoDB (local installation or MongoDB Atlas account)

Installation Steps

1. Clone the repository and navigate to the project folder.

```
git clone https://github.com/B-Vijayendra/customer-registry.git
cd customer-care
```

2. Install backend dependencies and configure environment variables.

```
cd server
npm install
cp .env.example .env
```

Edit the .env file to set MONGO_URI (local or MongoDB Atlas connection string) and JWT_SECRET.

3. Install frontend dependencies.

```
cd ../client
npm install
cp .env.example .env
```

4. Seed the database with default categories and an admin account (from the server folder).

```
node seed.js
```

5. Folder Structure

Client (client/src)

- components — reusable UI elements (Navbar, Sidebar, ComplaintTable, ChatBox, StatCard, etc.)
- pages — route-level views (Login, Register, Dashboard, Complaints, admin/* pages, etc.)
- layouts — DashboardLayout and AuthLayout wrappers
- routes — AppRoutes.jsx and ProtectedRoute.jsx for role-based routing
- services — one Axios-based service per API resource (authService, complaintService, etc.)
- context — AuthContext, ThemeContext, ToastContext
- hooks — useAuth, useFetch

Server (server/src)

- controllers — request handling logic per resource
- models — Mongoose schemas (User, Complaint, Category, Message, Notification, etc.)
- routes — Express route definitions per resource
- middleware — auth.js (JWT + role checks), errorHandler.js, upload.js
- config — db.js (MongoDB connection)
- uploads — stores complaint attachment files

6. Running the Project

Backend (http://localhost:5000)

```
cd server
npm install
npm run dev
```

Frontend (http://localhost:5173)

```
cd client
npm install
npm run dev
```

Once both servers are running, open <http://localhost:5173> in the browser. New users can register as a Customer or Agent from the landing page; the seeded Admin account can log in directly to manage users, agents, and categories.

7. API Documentation

The table below lists the major API endpoints exposed by the backend. All protected routes require an Authorization: Bearer <token> header.

Method	Endpoint	Access
POST	/api/auth/register	Public
POST	/api/auth/login	Public
GET / PUT	/api/auth/profile	Authenticated
POST	/api/complaints	Customer
GET	/api/complaints	Authenticated (scoped by role)
GET / PUT / DELETE	/api/complaints/:id	Owner / Assigned Agent / Admin
GET	/api/categories	Authenticated
POST / PUT / DELETE	/api/categories	Admin
GET	/api/users	Admin
GET	/api/users/analytics	Admin
POST / GET	/api/messages	Complaint participants only
GET / PUT	/api/notifications	Authenticated

8. Authentication

Authentication is implemented using JSON Web Tokens (JWT). On successful login, the server issues a signed token containing the user's ID, which the client stores and attaches to subsequent requests as a Bearer token. Passwords are hashed using bcryptjs before being stored in the database.

Protected routes are guarded by a protect middleware that verifies the JWT and attaches the authenticated user to the request. Role-based access control is enforced through an authorize middleware, which restricts specific routes to one or more of the three roles: Customer, Agent, and Admin. On the frontend, a ProtectedRoute component and AuthContext manage session state and redirect unauthenticated or unauthorised users accordingly.

9. User Interface

The application features a responsive, glassmorphism-style interface built with Tailwind CSS, supporting both light and dark modes with persistent user preference. Each role has a tailored dashboard: customers view and raise complaints, agents manage assigned complaints and chat with customers, and admins

access analytics charts alongside user, agent, and category management screens. Complaint records are presented in searchable, filterable, and paginated tables, with a dedicated detail view showing the complaint timeline, attachment, and chat thread. In-app notifications alert users to status changes and new assignments.

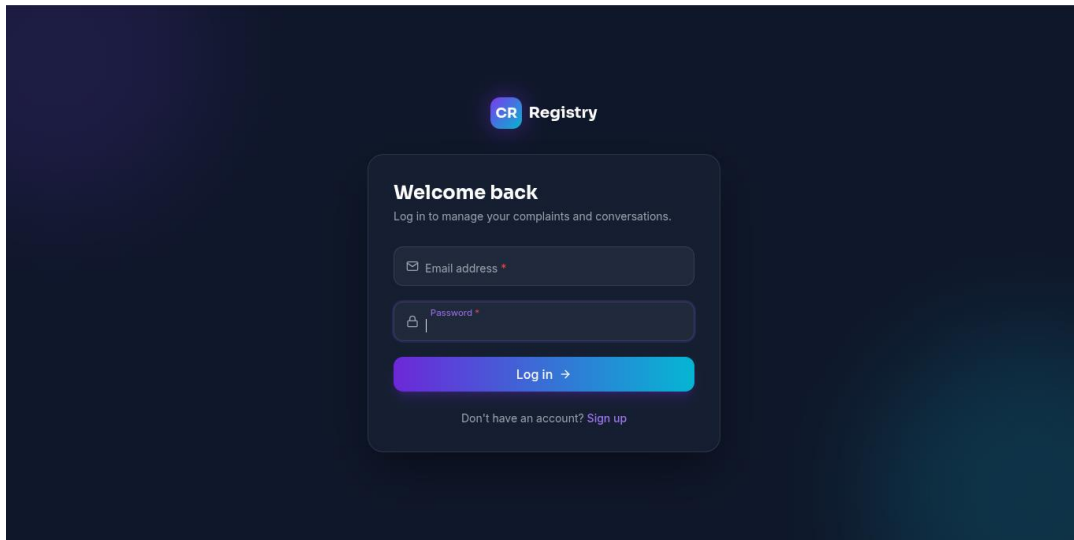


Figure 1: Login Page

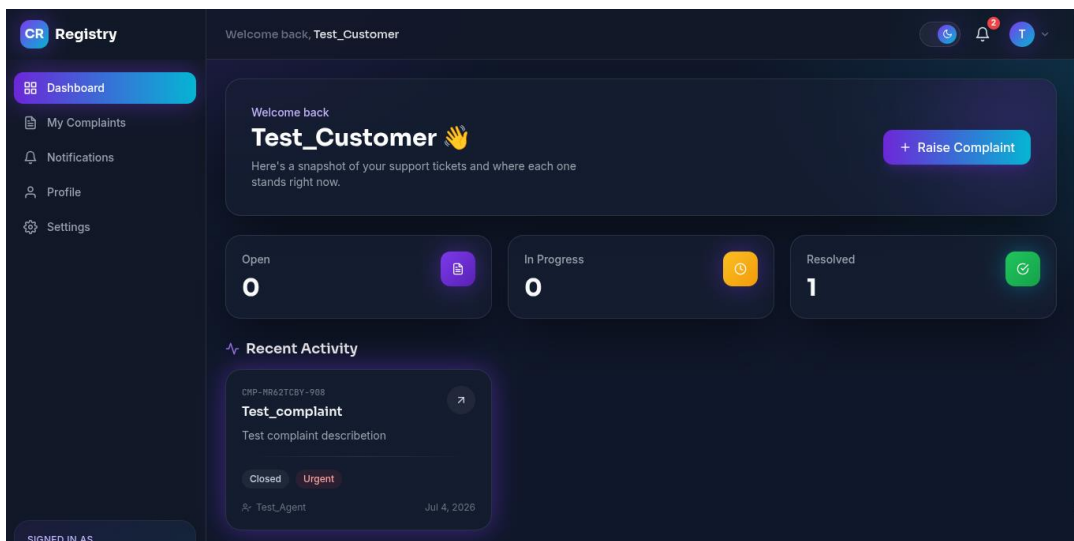


Figure 2: Customer Dashboard

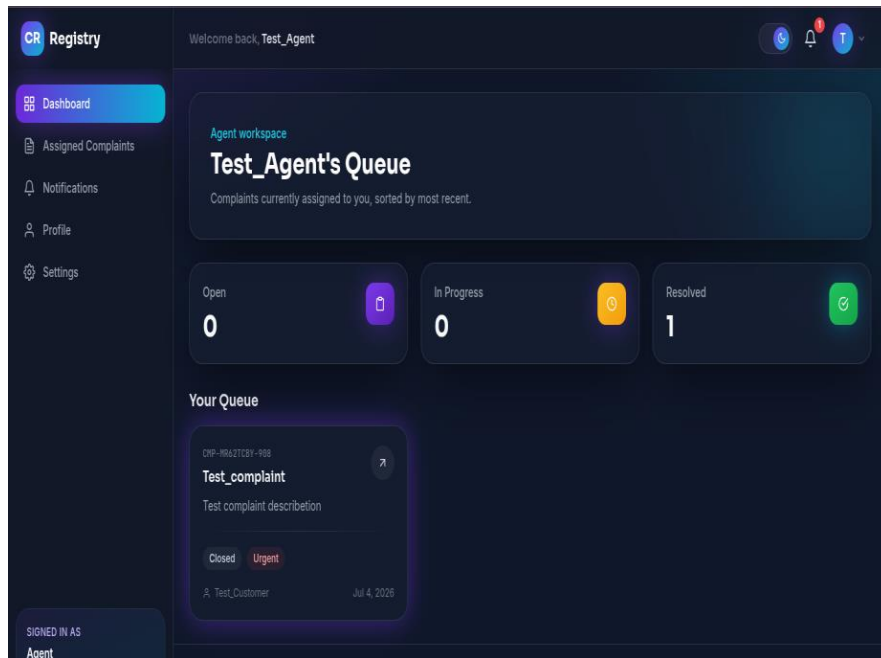
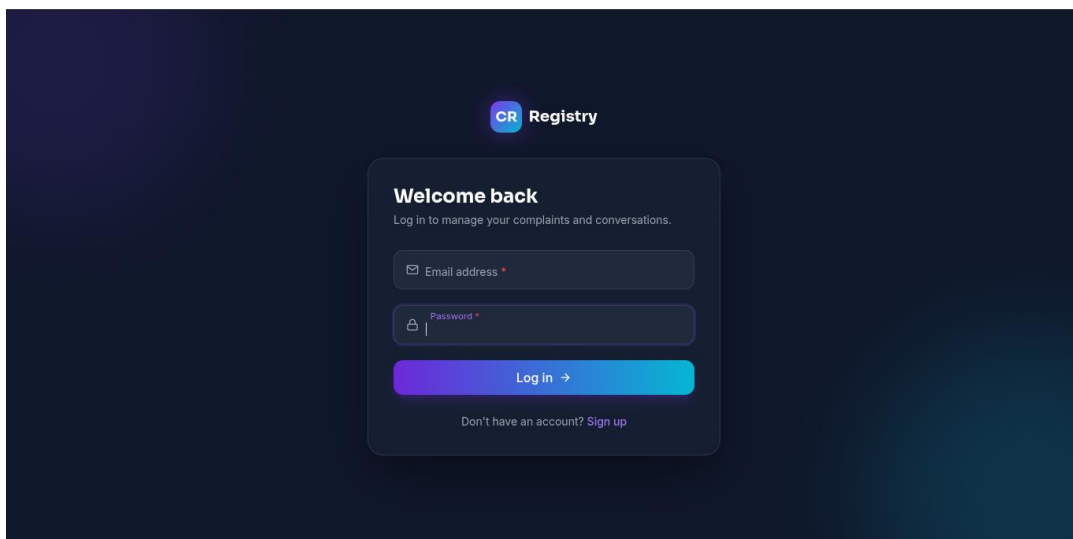


Figure 3: Admin Dashboard

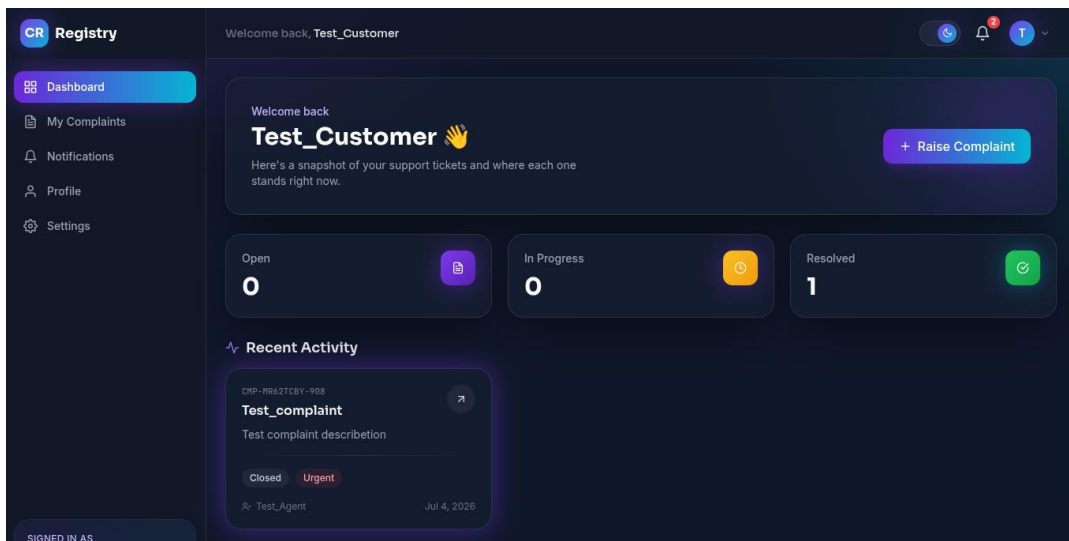
10. Testing

Testing was carried out manually using a structured checklist covering functional, API, and UI aspects. This included verifying role-based registration and dashboard access, complaint creation with and without attachments, agent assignment and notification delivery, status updates and timeline recording, chat messaging between customer and agent, feedback submission after resolution, search/filter/pagination on the complaints list, dark mode persistence, and route protection for unauthenticated and unauthorised access attempts.

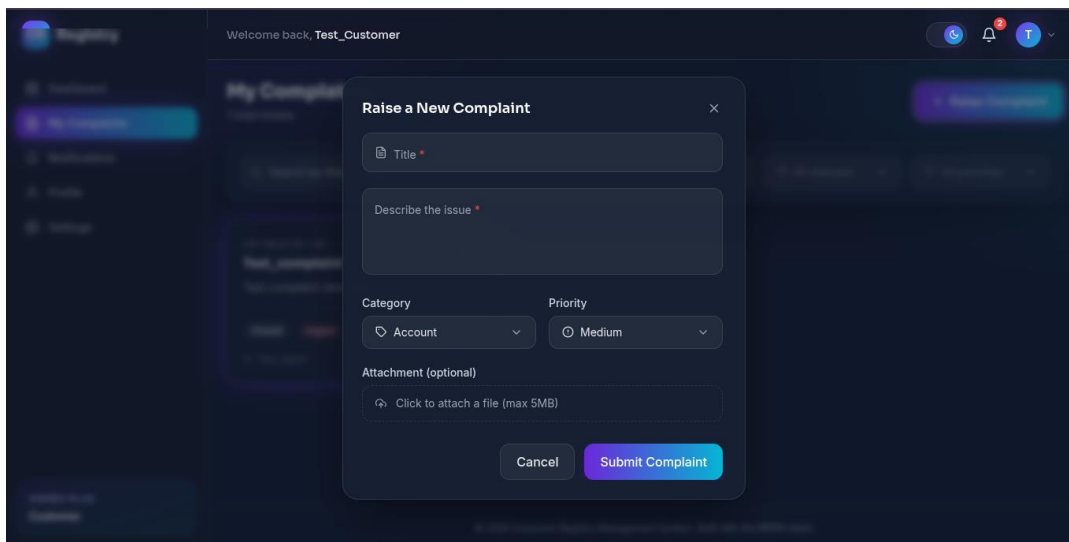
11. Demo / Screenshots



Login Page



Customer Dashboard



Raise a New Complaint

12. Known Issues

- Requires an active internet connection when using MongoDB Atlas.
- Complaint attachments are limited by the configured file upload size.
- Chat is implemented via REST polling (8-second interval) rather than real-time WebSockets, so messages may take a few seconds to appear.
- No email notification support; all notifications are in-app only.

13. Future Enhancements

- Email notifications for complaint updates and assignments.
- Real-time chat using Socket.IO instead of polling.

- AI-based automatic complaint categorisation.
- Enhanced analytics dashboard with additional metrics and filters.
- Mobile application version.
- Multi-language support.